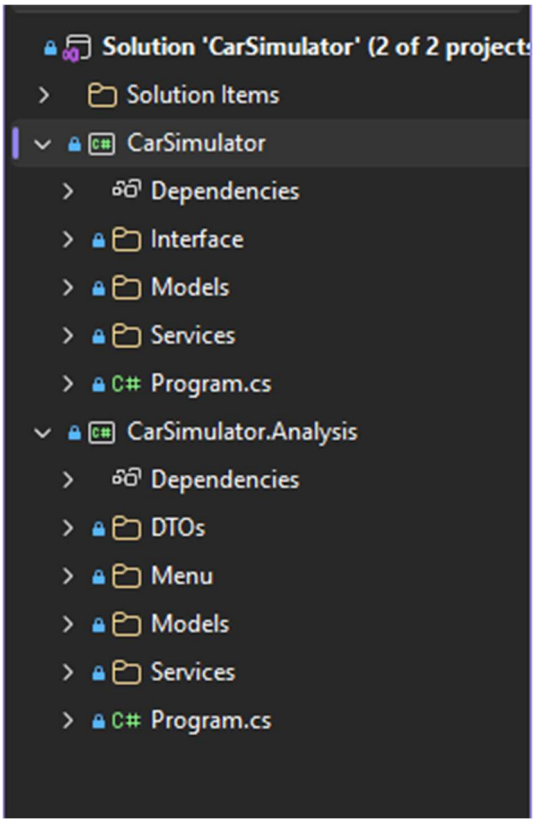
Inlämningsuppgift IoT

CarSimulator – Simulering och analys av biltelemetri

1. Inledning

Lösning och syfte



I detta projekt har jag utvecklat ett system som simulerar dataflödet från en uppkopplad bil. Syftet är att visa hur driftsdata kan genereras, skickas till en molnplattform och sedan hämtas för analys.

Lösningen består av två separata konsolapplikationer utvecklade i C# med .NET 10 i Visual Studio.

CarSimulator simulerar bilkörning och skickar motorvarvtal, hastighet, motortemperatur och bränslenivå till ThingSpeak.

CarSimulator.Analysis hämtar avläsningarna och beräknar genomsnittlig hastighet och genomsnittligt motorvarvtal.

Simuleringslogiken bygger på lärarens exempelprojekt IoT_CarSimulator. Koden har anpassats till min projektstruktur och kompletterats med molnkommunikation, analys, felhantering och möjlighet att avbryta simuleringen med Esc.

2. Design och arkitektur

Lösningens arkitektur

Lösningen är uppdelad i två projekt som kommunicerar via samma ThingSpeak-kanal. Projekten kan köras oberoende av varandra. Analysappen kan därför läsa tidigare sparade data även när simulatorn är avstängd.

I simulatorprojektet ansvarar **CarSimulationService** för bilens tillstånd och simuleringens körning. Modellen **CarTelemetry** innehåller en avläsnings värden och tidsstämpel. **ThingSpeakService** ansvarar för att skicka avläsningen till molnet.

I analysprojektet ansvarar en separat **ThingSpeakService** för hämtningen. **ThingSpeakResponseDto** och **ThingSpeakFeedDto** representerar JSON-svaret från API. **TelemetryAnalysisService** kontrollerar värdena och beräknar statistiken, som samlas i modellen **TelemetryStatistics**. **AnalysisMenu** hanterar menyval och presentation.

I båda projekten läser **Program.cs** konfigurationen, skapar tjänsterna och startar programmet. Uppdelningen gör det lättare att följa koden och ändra en del utan att behöva ändra hela programmet.

Dataflödet består av följande steg:

1. Simulatorn uppdaterar bilens tillstånd och skapar en avläsning.

2. Avläsningen skickas till ThingSpeak via HTTPS.
3. ThingSpeak lagrar värdena i kanalens fält.
4. Analysappen hämtar avläsningar för användarens val.
5. Värdena kontrolleras och statistiken visas i konsolen.

Jämförelse mellan ThingSpeak, Azure IoT Hub och AWS IoT

ThingSpeak erbjuder lagring, visualisering och analys av telemetri i kanaler. Azure IoT Hub och AWS IoT Core erbjuder fler möjligheter för enhetskommunikation och integration med andra molntjänster. För en motsvarande lösning med lagring och analys behöver dessa tjänster vanligtvis kombineras med ytterligare komponenter.

Jag valde ThingSpeak eftersom det var enklare och smidigare att komma igång med för denna uppgift. Jag kunde snabbt skapa en kanal och börja skicka och läsa data med API-nycklar.

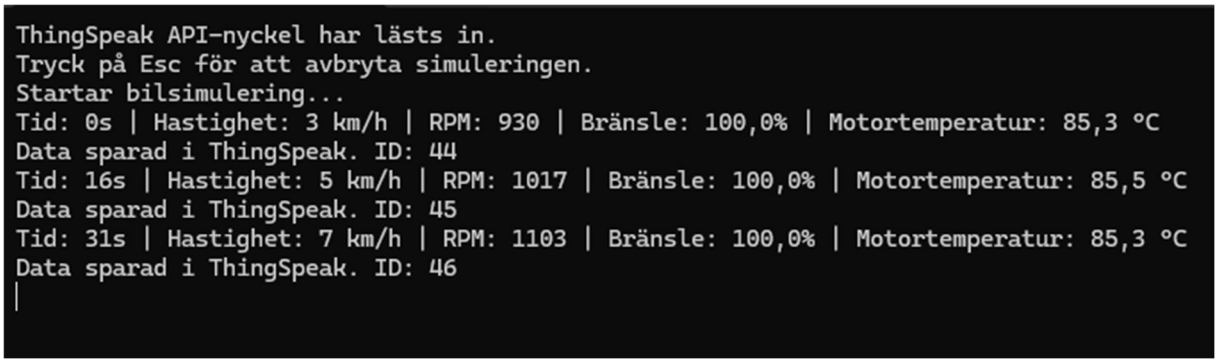
Azure IoT Hub upplevde jag som mer omständligt, när jag provade använda det för uppgifter, och det var till en början oklart vilka abonnemang och kostnader som krävdes. Det finns dock en begränsad kostnadsfri nivå för testning av Azure IoT Hub.

AWS IoT Core är också ett alternativ för att ansluta enheter och vidarebefordra meddelanden till andra molntjänster. Min jämförelse med AWS är översiktlig, eftersom AWS inte används i den färdiga lösningen och pga. Tidsbrist inte hunnit testa det för några uppgifter.

ThingSpeak passade projektets omfattning och gjorde att jag kunde fokusera på C#-applikationerna och dataflödet.

3. Implementering

Simulering, integration och automatisering

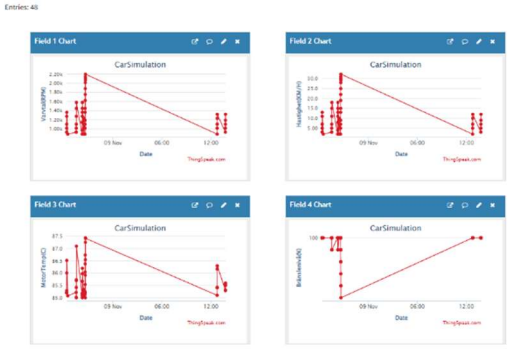


Simuleringen utgår från lärarens metoder **StartSimulation**, **SimulateTrip**, **CalculateRpm** och **GenerateEngineTemperature**. Hastigheten förändras beroende på aktuell körfas, medan varvtalet beräknas utifrån hastigheten. Bränslenivån minskar under körningen och motortemperaturen genereras utifrån hastigheten.

Simuleringen körs automatiskt i cirka tio minuter om användaren inte avbryter den. Efter varje sändningsförsök väntar programmet 15 sekunder innan nästa avläsning skapas. Tiden för själva nätverksanropet tillkommer.

Avläsningarna skickas med HttpClient till ThingSpeaks API. Fälten är kopplade enligt följande:

ThingSpeak-fält	Värde
Field 1	Motorvarvtal
Field 2	Hastighet
Field 3	Motortemperatur
Field 4	Bränslenivå



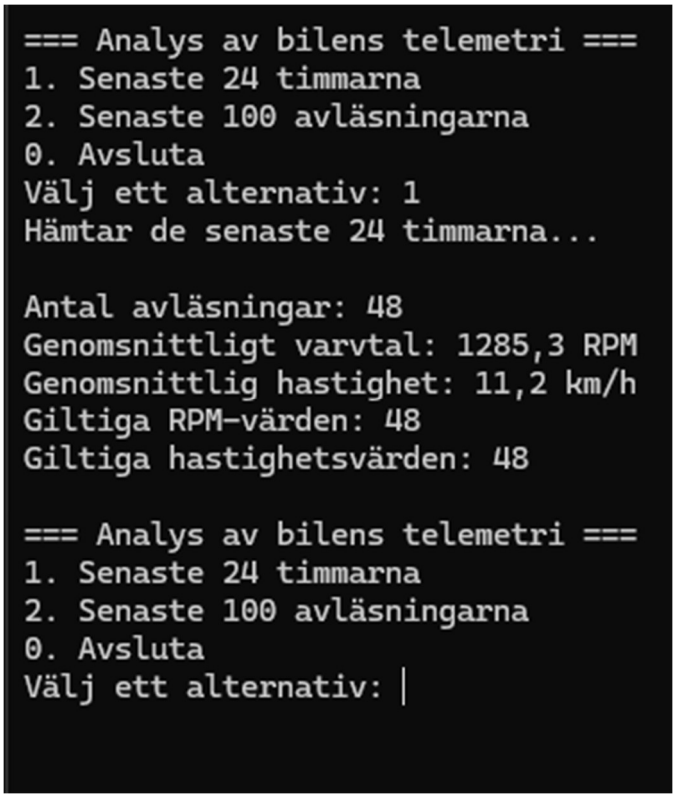
1

API-nyckeln och värdena skickas i anropets

innehåll. Programmet kontrollerar både HTTP-status och ThingSpeaks svar. Ett positivt post-ID visar att avläsningen har sparats.

Analys och statistik

Analysappen har en meny där användaren väljer de senaste 24 timmarna eller de senaste 100 avläsningarna. Svaret hämtas som JSON och omvandlas till DTO-objekt med System.Text.Json.



ThingSpeak returnerar fältvärden som text. Därför omvandlas de till numeriska värden innan beräkningen. Saknade, ogiltiga och orimliga värden utesluts. Hastighet och varvtal kontrolleras var för sig, så att ett saknat hastighetsvärde inte gör ett giltigt varvtal oanvändbart.

Programmet visar medelvärden och hur många giltiga värden som ingår i respektive beräkning. Om inga giltiga värden finns visas ett meddelande i stället för ett missvisande medelvärde på noll.

Centrala tekniska lösningar

Asynkron kommunikation: Nätverksanrop och väntan använder async och await. Exempelvis inväntas sändningen innan nästa väntetid börjar:

await _thingSpeakService.SendTelemetryAsync(data);

Avbrytning:

När användaren trycker på Esc signalerar en CancellationTokenSource att simuleringen ska avslutas. Väntan mellan avläsningarna kan då avbrytas. Ett redan pågående HTTP-anrop får avslutas eller nå sin timeout.

```
// Skapa en signal som kan stoppa simuleringen.
using var cancellation = new CancellationTokenSource ();

Console.WriteLine ("Tryck på Esc för att avbryta simuleringen.");

// Starta simuleringen och kontrollera tangentbordet medan den körs.
var simulationTask = simulator.StartSimulation (cancellation.Token);

while (!simulationTask.IsCompleted)
    {
    if (Console.KeyAvailable &&
        Console.ReadKey (intercept: true).Key == ConsoleKey.Escape)
        {
        Console.WriteLine ("Avbryter simuleringen...");
        cancellation.Cancel ();
        break;
        }

    // Kontrollera tangentbordet utan att belasta processorn i onödan.
    await Task.Delay (100);
    }

// Låt simuleringen avslutas innan programmet fortsätter.
await simulationTask;
```

Talformat: CultureInfo.InvariantCulture används vid överföring och tolkning av tal. Det gör att decimalpunkt hanteras konsekvent även om datorns språk använder decimalkomma.

Tidsstämplar: Avläsningarna tidsstämplas i UTC för att undvika att lagringen beror på datorns lokala tidszon.

Säkerhetsåtgärder och felhantering

API-nycklarna hämtas från User Secrets och skrivs inte direkt i källkoden. Simulatorn använder kanalens skrivnyckel, medan analysappen använder läsnyckeln och kanalens ID.

Programmen kontrollerar konfigurationen vid start. Nätverksfel, timeout och misslyckade API-anrop hanteras med felmeddelanden. Analysappen hanterar även felaktiga JSON-svar, tomma resultat och ogiltiga menyval.

Vid manuella tester kontrollerades menyvalen, överföringen till ThingSpeak och visningen av statistik. Analysappen testades även med frånkopplat internet och fel läsnyckel. Den visade felmeddelanden och återgick till menyn. Efter återställning av anslutningen respektive nyckeln gick det att hämta data igen.

4. Säkerhetsanalys och åtgärder

Risker och konsekvenser

Risk	Möjlig konsekvens	Åtgärd eller begränsning
API-nycklar läcker	Obehöriga kan läsa privata data eller lägga till falska avläsningar.	Nycklar lagras i User Secrets och lämnas separat vid granskning.
Obehörig åtkomst till datorn	Lokalt lagrade nycklar kan läsas.	User Secrets är inte krypterat och kräver att datorn och användarkontot skyddas.
Nätverksavbrott	Avläsningar når inte molnet och statistiken kan bli ofullständig.	Felhantering gör att programmet kan fortsätta, men oskickade avläsningar sparas inte lokalt.
Felaktiga eller manipulerade värden	Medelvärden kan bli missvisande.	Analysen kontrollerar format, numeriska gränser och saknade värden. Rimliga men falska värden kan fortfarande passera.
Felaktig systemtid	Avläsningar kan få fel tidsstämpel och hamna utanför vald period.	UTC används, men datorns klocka behöver fortfarande vara korrekt.

HTTPS skyddar kommunikationen under överföringen. Det skyddar däremot inte mot att någon får tillgång till API-nycklarna på datorn eller genom att de av misstag publiceras.

Lösningen är anpassad för ett utbildningsprojekt. För en produktionslösning skulle säkrare hantering av hemligheter, övervakning och lokal lagring vid avbrott behöva utvecklas vidare.

5. Slutsats och reflektion

Projektet visar ett fungerande dataflöde från simulerad biltelemetri till molnlagring och vidare till analys i en separat applikation. Uppdelningen i modeller, DTO-klasser, tjänster och meny gör ansvarsfördelningen tydlig.

En utmaning var att välja en molnplattform och förstå hur anslutning och åtkomst skulle konfigureras. ThingSpeak gjorde det enklare att komma igång. Arbetet tydliggjorde också skillnaden mellan en skrivnyckel för att skicka data och en läsnyckel för att hämta data.

Jag undersökte även möjligheten att köra simulering och analys från en gemensam meny. När båda skrev till samma konsolfönster blev presentationen rörig. Jag valde därför att behålla två separata konsolapplikationer, som kan köras samtidigt i varsitt fönster.

Genom arbetet har jag fått bättre förståelse för asynkrona anrop, JSON, DTO-klasser, API-kommunikation och avbrytning med CancellationToken. Jag har också arbetat med att skilja mellan saknade data och faktiska nollvärden, vilket är viktigt för rättvisande statistik.

Lösningen kan vidareutvecklas med lokal buffring och återförsök vid misslyckade sändningar. Andra förbättringar är automatiserade tester av statistikberäkningarna, mer realistisk körsimulering samt stöd för felkoder, GPS-positioner och kartvisning.

Källor

- Lärarens exempelprojekt: https://github.com/CKV-SM/IoT_CarSimulator
- ThingSpeak: https://thingspeak.mathworks.com/
- ThingSpeak – skriva data: https://www.mathworks.com/help/thingspeak/writedata.html
- ThingSpeak – läsa data: https://www.mathworks.com/help/thingspeak/readdata.html
- Inlämning Git repo : https://github.com/Siyament-lab/CarSimulator.git